Инструкции и ответы языковой модели

Как управлять продолжением и проверять результат

Связный ответ нужно сверить с вопросом

Слайд 1. Инструкции и ответы LLM

Студент спрашивает, можно ли продлить книгу, которую уже забронировал другой читатель. Модель отвечает коротко и уверенно: «Можно». Такой ответ легко прочитать, но пока неизвестно, верен ли он. В документе нашей учебной библиотеки сказано обратное: бронь другого читателя делает продление недоступным.

На четвёртой лекции языковая модель выбирала следующие токены. Теперь её продолжение выглядит как ответ помощника. Способ предсказания по-прежнему связан с вероятностями, а прикладная задача требует дополнительных свойств: ответить именно на вопрос, сохранить условие из источника и выполнить заданный формат.

Слайд 2. Один документ, три проверки

Возьмём один документ D03. Он разрешает продление на 14 дней через кабинет читателя, но запрещает его при чужой брони. Число продлений за семестр в документе не задано. По этим сведениям можно составить три разных проверки: срок, исключение и вопрос без нужного факта.

Сначала сами ответим на них по документу. Для срока нужен ответ «14 дней». Для брони — «нет», потому что действует условие. Для числа продлений нужно признать отсутствие сведений. Эти ожидания определены до обращения к модели. Иначе красивый ответ легко принять за подходящий только потому, что он звучит правдоподобно.

В лабораторной используется Qwen3-0.6B с фиксированными весами. Мы будем менять инструкцию и способ выбора токенов, наблюдая ответы на одной и той же информационной основе. Результаты дадут возможность отдельно обсудить вычисление распределения, формулировку задачи и проверку содержания.

Почему модель умеет отвечать на инструкцию

Слайд 3. Обучающие сигналы различаются

При авторегрессионном предобучении модель учится продолжать тексты. Она получает префикс и повышает вероятность следующих токенов из обучающего материала. Это помогает освоить закономерности языка и содержание данных, но задача сама по себе не требует отвечать пользователю в нужном формате.

Обучение на демонстрациях инструкций, supervised fine-tuning или SFT, использует пары «запрос — желаемый ответ». Например, на просьбу кратко объяснить термин дано подходящее объяснение. Параметры изменяются так, чтобы соответствующее продолжение получало большую вероятность при данном запросе и формате диалога.

Другая группа методов использует предпочтения между ответами. Для одной инструкции представлены варианты A и B, и указано, какой лучше по выбранным критериям. RLHF может включать модель награды и оптимизацию поведения по этой награде. DPO использует пары предпочтений непосредственно в целевой функции. Для нас сейчас важнее различие обучающего сигнала, чем детали оптимизаторов.

Предпочтение зависит от критериев оценивания. Более длинный ответ с выдуманным ограничением «не больше двух продлений за семестр» может выглядеть обстоятельнее короткого отказа. Если оценщик не сверяет документ, он способен предпочесть неподдержанный ответ. Поэтому качество стиля и наличие фактического основания следует различать и при обучении, и при нашей проверке.

Эти этапы не являются обязательной одинаковой цепочкой для всех моделей. Конкретную историю обучения описывает автор checkpoint. Слово «инструктивная» говорит о назначении модели, но не обещает безошибочного выполнения любой новой просьбы.

В нашей работе веса не меняются. Добавленная инструкция влияет на текущий вход, а не запускает SFT. Это различие станет особенно наглядным, когда мы добавим в контекст несколько примеров решения.

Инструкция делает ожидания проверяемыми

Слайд 4. Что именно требуется от ответа

Запрос «Помоги студенту с библиотекой» оставляет много свободы. Неясно, какой вопрос нужно решить, разрешено ли добавлять общие советы, нужна ли короткая фраза и что делать, если сведений нет. Для свободной беседы такая постановка может подойти, но сравнить результаты по ней трудно.

Разделим вход на инструкцию, документ и вопрос. В базовом варианте просто передадим документ и спросим о продлении. В уточнённом добавим: «Используй только документ ниже. Ответь одним коротким предложением. Если нужного факта нет, ответь точно: НЕТ ДАННЫХ».

Теперь можно проверить несколько отдельных требований. Отвечает ли текст на поставленный вопрос? Поддерживает ли документ главный факт? Сохранено ли исключение про бронь? Появились ли дополнительные правила, которых в источнике нет? Выполнен ли требуемый отказ, когда число продлений неизвестно?

Инструкция выражает желаемое поведение, а реальный вывод ещё предстоит прочитать. Фраза «используй только документ» не исключает автоматически ошибочное число или неверное условие. Благодаря одному короткому источнику нам будет удобно проверить это вручную.

Слайд 5. Две инструкции, один источник

При сравнении оставим сам документ и вопросы одинаковыми. Если вместе с уточнённой инструкцией добавить новый источник, разницу ответа уже нельзя связать только с инструкцией. Это тот же принцип контролируемого изменения, который применялся при сравнении словных и символьных признаков.

Формат выбирается по потребителю ответа. Для человека полезна короткая фраза. Для программы можно потребовать JSON с полями `answer` и `has_answer`. Тогда появится точная проверка структуры. Но корректный объект с неверным значением остаётся неверным ответом.

Примеры в контексте показывают способ ответа

Слайд 6. Пример добавляется в контекст

В zero-shot постановке мы даём инструкцию без решённых примеров текущей задачи. В few-shot добавляем несколько пар «вход — нужный выход». Например, один пример показывает, как кратко вернуть факт из документа, второй — как ответить при его отсутствии.

Представим учебное правило: «Ссылка для восстановления пароля действует 30 минут». На вопрос о времени действия образец возвращает «30 минут». На вопрос о числе повторных запросов образец возвращает «НЕТ ДАННЫХ», если это число не указано. Такие примеры задают и содержание задачи, и желаемую форму ответа.

Слайд 7. Few-shot не обновляет веса

Параметры от этих демонстраций не обновляются. Модель читает их как часть текущего контекста. Если в следующем независимом вызове примеров нет, приложение не может рассчитывать, что они стали постоянным знанием весов.

История чата тоже передаётся приложением, а не возникает внутри отдельного запроса сама собой.

Примеры занимают место во входе. Их полезность следует сравнивать с ценой: можно потратить доступный контекст на почти одинаковые демонстрации и вытеснить важный фрагмент нового документа. Для нашего основного опыта few-shot не добавляется — мы сравниваем два варианта инструкции при одном источнике.

Если в демонстрациях допущена ошибка, она может направлять новый ответ в ту же сторону. Поэтому полезно выбирать случаи, которые показывают границы задачи: положительный факт, исключение, отсутствие сведений. Один набор только удачных простых вопросов не объясняет, как поступать в сложном случае.

Temperature меняет концентрацию

Слайд 8. Масштабирование оценок

Пусть модель уже вычислила оценки следующего токена z. Для выборки можно разделить их на положительную температуру T, а затем применить softmax: $p_i=\exp(z_i/T)/\Sigma_j\exp(z_j/T)$. Это изменение распределения выбора, а не переобучение модели.

Слайд 9. Три значения temperature

Посчитаем три оценки (2;1;0). При T=1 вероятности примерно (0,6652;0,2447;0,0900). При T=0,5 различия оценок усиливаются: получается (0,8668;0,1173;0,0159). При T=2 распределение более ровное: (0,5065;0,3072;0,1863).

Порядок трёх оценок при положительной T сохраняется. Меняется величина преимущества первого токена по вероятности. Поэтому при sampling первый вариант будет встречаться чаще при низкой температуре, но по одному короткому ответу при каждой температуре это различие может быть незаметно.

Для устойчивого вычисления softmax вычитаем максимум из оценок перед экспонентой. Общий сдвиг не меняет распределение, как мы проверяли в attention. Здесь нормируется одна строка альтернатив следующего шага.

Слайд 10. Уверенность и достоверность

Greedy в нашем коде задаётся выбором максимума и отключённой выборкой. Мы не делим оценки на T=0: это не определённая математическая операция. При фиксированных оценках greedy вообще не нуждается в вероятностной выборке из оставшихся альтернатив.

Высокая температура не измеряет творческие способности, а низкая — достоверность. Если неверный ответ имеет наибольшую оценку, более сосредоточенное распределение не добавит в него недостающий факт. Вопрос правильности проверяется по источнику отдельно от параметров декодирования.

Top-p выбирает набор кандидатов

Слайд 11. Top-p оставляет нужную массу

Nucleus sampling, или top-p, сначала упорядочивает вероятности по убыванию. Затем оставляет минимальное число верхних элементов, накопленная сумма которых достигает порога p. Остальные исключаются, а оставшиеся вероятности нормируются заново.

Для распределения (0,5;0,3;0,2) и p=0,75 одного элемента недостаточно: его масса 0,5. Два дают 0,8 и пересекают порог.

Слайд 12. Делим на выбранную массу

Делим первые два числа на 0,8 и получаем (0,625;0,375;0). Третий токен больше не участвует в выборке.

Число сохранённых элементов меняется с распределением. Если один вариант уже имеет вероятность 0,9 при p=0,8, достаточно его одного. Если масса распределена между многими вариантами, набор станет шире. В этом отличие от top-k, который ограничивает число верхних кандидатов заранее.

Слайд 13. Порядок правил декодирования

В реальном опыте дополнительно задан top_k=20. В используемой версии библиотеки для этих параметров сначала применяется temperature, затем top-k и top-p. Поэтому реальную выборку нельзя объяснять только операцией top-p, которую мы отдельно разобрали на числах. Упрощённая функция объясняет одну операцию; полный вызов сохраняет все настройки.

Слайд 14. EOS или предел продолжения

Ограничение новых токенов решает другую задачу. `max_new_tokens=64` задаёт предел числа новых предсказаний; оно не означает, что ответ обязан иметь 64 обычных слова. Модель может раньше выдать EOS, а может достичь предела, не завершив мысль. Причина остановки будет сохранена вместе с исходным ответом.

Нужно учитывать и длину входа. Инструкция, документ, вопрос и специальные токены диалогового шаблона занимают места общего контекста. Сначала

измеряется фактический вход после шаблона, затем проверяется, помещается ли он вместе с запрошенным продолжением в окно модели.

Шаблон диалога входит в запрос

Слайд 15. От ролей к последовательности

В интерфейсе мы видим сообщения с ролями system, user и assistant. Для модели они превращаются в последовательность специальных разделителей и содержимого. Эту процедуру задаёт chat template, связанный с выбранным токенизатором и моделью.

В лабораторной общее system-сообщение просит отвечать по-русски. Документ, вопрос и сравниваемая инструкция находятся в user-сообщении. Это важно для устройства опыта: ограничение «только по документу» добавлено лишь в strict-варианте, а не спрятано в общем system для обоих вариантов.

Слайд 16. Считаем весь вход модели

Вызов `apply_chat_template` использует `add_generation_prompt=True`, чтобы обозначить начало ответа assistant. Для выбранного Qwen3 задан `enable_thinking=False`. Если сначала получаем от шаблона строку, а потом токенизируем отдельно, ставим `add_special_tokens=False`, чтобы не добавить специальные позиции второй раз.

Функция подготовки возвращает пакет тензоров и его реальную длину. Пустой ввод или вход длиннее установленного в опыте предела 768 токенов отклоняется. Документ не усекается молча: иначе сравниваемые инструкции могли бы работать с разными остатками источника.

Слайд 17. Отделяем новые токены

У decoder-only генератора выход обычно включает исходный префикс и новые токены. Поэтому после генерации берём `output[0, input_length:]` и декодируем только эту часть. Если показать весь массив как ответ, рядом окажутся и введённый документ, и собственный вывод модели.

Сохраняем исходный user-текст, режим, параметры выборки, seed, длины и причину остановки. Это позволяет повторить сравнение и понять, не изменился ли вместе с инструкцией сам вход. Пунктуация, переносы и порядок частей тоже относятся к нему.

Факт, формат и отказ проверяются отдельно

Слайд 18. Два критерия ответа

Рассмотрим ответ «Можно» на вопрос о книге с чужой бронью. Он короткий, но противоречит D03. Теперь ответ «На 14 дней» на вопрос о количестве продлений за семестр. Сам срок подтверждён документом, однако он отвечает на другой вопрос. Эти ошибки различаются, хотя обе требуют нулевой оценки главного факта поставленной задачи.

В нашей ручной рубрике главный факт получает 2 балла, если ответ верен и отвечает на вопрос; 1 — если он частичен, но не содержит противоречия в главном факте; 0 — если главный ответ неверен, отсутствует или подменяет вопрос. Рядом отдельно записываются неподдержанные утверждения и пояснение по документу.

Слайд 19. Смысл отказа и формат

Для strict проверяется и формат. Если данных нет, требуется точная строка «НЕТ ДАННЫХ». «В источнике это не указано» может быть содержательно подходящим отказом, но не совпадает с заданным форматом. Для basic такого требования не было, поэтому несовпадение с этой строкой нельзя считать нарушением инструкции.

Слайд 20. Схема верна, факт ошибочен

В JSON-опыте требуются ровно два ключа: `answer` со строкой и `has_answer` с логическим значением. Начнём с обычного JSON-парсера, затем проверим корневой тип, набор ключей и типы значений. Строка `"false"` не равна логическому `false`. Markdown-ограждение вокруг объекта тоже не входит в заданный ответ.

Валидатор не дописывает оборванную структуру и не удаляет ограждения за моделью. Иначе мы измерили бы уже исправленный приложением результат. Исправления можно исследовать отдельно, но сначала нужно увидеть исходное соблюдение требований.

Даже после успешного парсинга остаётся проверка фактов. Объект `{"answer":"42","has_answer":true}` может полностью соответствовать схеме, но противоречить документу о 14 днях. Синтаксис и содержание проверяются разными действиями.

Что показал опыт с одним документом

Слайд 21. Условие прочитано неверно

В основной части выполнены шесть запросов: две инструкции и три вопроса, всегда greedy. Оба варианта правильно назвали срок 14 дней. Оба ошиблись на условии чужой брони. В strict модель вернула «Ответ: Можно.» — короткая форма не исправила главный факт.

Слайд 22. Срок не заменяет число раз

На вопрос о числе продлений basic начал добавлять правило о продлении каждую неделю и дошёл до предела 64 новых токенов. Это неподдержанная подробность и незавершённый ответ. Strict вместо количества снова назвал срок 14 дней: «Продление книги можно продлить на 14 дней.». Здесь не было нужного отказа, хотя связанная с темой цифра оказалась правильной сама по себе.

Слайд 23. Четыре одинаковых ответа

Затем один strict-промпт о сроке проверен при T=0,5 и T=1 с seed 7 и 42, при одинаковых top-p и top-k. Все четыре выборки дали «14 дней.» и завершились EOS. В этих наблюдениях изменение параметров не проявилось в видимом тексте. Это не означает, что распределение осталось прежним: его изменение мы уже рассчитали на числовом примере.

Слайд 24. JSON прошёл обе проверки

Отдельный JSON-запрос вернул `{"answer":"14","has_answer":true}`. Структура прошла проверку, число соответствует вопросу о днях и документу. В схеме не требовалось включать слово «дней» внутрь строки, поэтому его отсутствие не является форматной ошибкой этого опыта.

Вывод сформулируем по наблюдениям. Уточнение инструкции сохранило правильный срок, но не обеспечило чтение исключения и отказ при недостатке данных. На выбранном вопросе четыре настройки выборки дали одинаковый текст. JSON получился корректным в одном проверенном случае. Каждый из этих результатов относится к своему небольшому сравнению.

Для коллекции документов понадобится поиск

Слайд 25. Следующий шаг — поиск

Сегодня документ был известен заранее и целиком передан модели. Поэтому ошибки нельзя объяснять тем, что поисковик не нашёл D03. Мы исследовали инструкцию, декодирование и чтение доступного источника.

В реальном помощнике пользователь не станет каждый раз прикладывать нужную инструкцию. База может содержать десятки или тысячи документов. Придётся сначала найти подходящие фрагменты, выбрать их для контекста и затем получить ответ. Это добавит новые причины успеха и ошибки.

На следующей лекции соединим поиск и генерацию в RAG. Сравнение с сегодняшним опытом будет полезным: если правильный документ уже был в контексте, одно улучшение поиска не исправит неверное чтение условия. Чтобы выбрать изменение системы, нужно знать, где именно возникла проблема.

Для дальнейшего чтения

– Ouyang et al. Training language models to follow instructions with human feedback — демонстрации и предпочтения.

– Rafailov et al. Direct Preference Optimization — обучение по парам предпочтений.

– Holtzman et al. The Curious Case of Neural Text Degeneration — nucleus sampling.

– Transformers: chat templates — представление диалога.

– Qwen3: описание авторов — семейство модели и режим шаблона.

Числовые распределения заданы для ручного расчёта. Ответы основного опыта сохранены без исправлений в журнале лабораторной; сведения D03 относятся к учебному университету «Север».